CS229S Project 2 实验报告：GPT-2 上的 KV Cache、CUDA Graph 与推测解码（CPU 原型 + RTX 5090 实测）

ShaneLiu04*

2026 年 10 月

摘要

本报告完成 Stanford CS229S（Fall 2023）Project 2 的两大部分：给 nanoGPT 风格的解码器加上 KV cache，以及实现 Leviathan 和 Chen 提出的推测解码（speculative decoding），测量加速并与理论对照。实验分两层。第一层是 CPU 原型：在合成语法上训练一个 4 层 target 和一个 1 层 draft，用来做严格的单元测试，包括 float64 下 KV 与无 cache 的 logits 差 $\leq 10^{-9}$，以及对接受-拒绝采样做 60,000 次卡方检验。第二层是 GPU 实测：我自己写的 GPT-2 前向直接加载 HuggingFace 的 gpt2、gpt2-medium、gpt2-large、gpt2-xl 真实权重，在 RTX 5090 上实现了无 cache、预分配静态 KV cache、CUDA Graph decode 三个版本，以及带 KV 回滚的 GPU 推测解码（贪心与采样两种）。

主要结论有六条。(a) 正确性：fp32 下 gpt2-large / gpt2-xl 的 logits 与 HF 的最大差为 4.2×10^{-5} / 4.6×10^{-5}；4 个模型、三种实现的贪心输出 60/60 逐 token 相同。(b) 延迟：gpt2-xl 在上下文 1024 时，每 token 延迟从无 cache 的 20.62 ms 降到 KV 加 CUDA Graph 的 4.22 ms。但 batch 1 decode 仍是 roofline 下界的约 1.9 倍，多出来的时间与层数成正比，相当于每层约 30–41 µs 的 kernel 固定开销。短上下文时，eager 模式的 KV cache 甚至比无 cache 更慢。(c) 显存：KV cache 的 `memory_allocated` 在 58/63 个配置上与公式逐字节相等。(d) 推测解码的加速：gpt2→gpt2-xl 贪心实测 1.631 倍，T=1 时 1.454 倍。接受率 $\hat{\alpha}$ 与 $1-\mathrm{TV}$ 吻合；采样时每次调用产出的 token 数与 $(1-\alpha^{\gamma+1})/(1-\alpha)$ 相差 $<1\%$；代入实测的 c=0.138、$v\approx$1.11，预测与实测之比为 0.87–1.00。(e) 无损性：fp32 下推测解码的贪心输出与 target 120/120 一致；采样版做 4000 次，对精确 target 分布的卡方 p 值为 0.657，对 draft 分布为 8.9×10^{-168}。(f) 失效条件：数据里出现了加速变成减速的三种方式，分别是 γ 过大（gpt2-large T=1, $\gamma\geq5$）、draft 过大（gpt2-medium 的 c=0.308）和 batch 过大（验证不再免费）。

目录

1 背景与作业要求 **3**

2 实验环境 **3**

3 方法与实现 **4**

3.1 GPT-2 前向与权重加载 4

3.2 静态 KV cache 与 CUDA Graph 4

3.3 GPU 推测解码 5

*GitHub: `https://github.com/ShaneLiu04`。代码与原始指标：`https://gitee.com/liu-xingyan04/mlsys-infra`。本文基于公开课程材料独立完成，与课程团队无隶属关系。

3.4 接受率 α 的定义 5
3.5 理论模型 5

4 实验结果 5
4.1 CPU 原型：KV cache 与推测解码（已有部分） 6
4.2 GPU 正确性：与 HF 对齐，贪心输出一致 7
4.3 每 token 延迟与上下文长度、模型大小的关系 8
4.4 Prefill 与 decode 9
4.5 吞吐与 batch：从 memory-bound 到 compute-bound 的拐点 10
4.6 KV cache 显存：实测与公式 12
4.7 推测解码：接受率与每次 target 调用的 token 数 12
4.8 推测解码：墙钟时间加速 14
4.9 采样版推测解码不改变 target 分布 15
4.10 推测解码在大 batch 下为什么失效 16

5 与理论模型的对照 17
5.1 decode 的 roofline 17
5.2 推测解码 17
5.3 分布无偏的证明 18
5.4 模型与实测的差距汇总 18

6 面试专题 18
6.1 20 秒电梯陈述 18
6.2 STAR 故事 18

7 局限 20

8 结论 20

1 背景与作业要求

大模型逐 token 生成时，每一步都要把全部权重从显存读一遍，却只算出一个 token，所以 decode 通常卡在带宽上，算力大量闲置。Project 2 的两个技术都是在这个前提下省时间：KV cache 避免重复计算前缀，推测解码让一次读权重产出多个 token。官方要求可以概括为三件事：

1. KV cache：在 nanoGPT 上实现 KV cache，比较有无 cache 的生成速度，并分析显存开销；
2. 推测解码 [1]：用小模型（draft）提议 γ 个 token，大模型（target）用一次 forward 验证，实现接受-拒绝 [2] 采样，测量接受率与加速比；
3. 分析：用 roofline [5] 或访存模型解释为什么 decode 受带宽限制（memory-bound）[4]，以及推测解码为什么在 GPU 上有效、什么时候失效。

表 1 把官方各部分对应到本仓库的实现。

表 1: 官方作业与本仓库实现的对应关系

官方部分	CPU 原型（`code/kvspec`）	GPU 实测（`gpu/`）
nanoGPT + KV cache	`model.py`：预分配 $(L, B, H, T_{\max}, d_h)$ 缓冲区，通过 `length` 回滚	`gpt2_fast.py`：同样的静态 cache，加载 HF GPT-2 [3] 权重；eager 与 CUDA Graph 两种 decode
正确性	float64 下 KV 与无 cache 的 logits 差 $\le 10^{-9}$	fp32 logits 对照 HF `transformers`；贪心输出逐 token 比较
推测解码	`speculative.py`：numpy 实现接受-拒绝	`SpecEngine`：接受-拒绝全在 GPU 上完成，每轮只同步一次
分布无偏	显式给定 p, q，做 60,000 次卡方检验	真实的 gpt2→gpt2-large，4000 次采样对精确 p 做卡方检验
理论对照	$(1-\alpha^{\gamma+1})/(1-\alpha)$，$c$ 用实测值	同左，另加 `mlsys_calc.decode_step_time` 的 roofline 预测
显存	公式	`torch.cuda.memory_allocated` 实测对照公式

2 实验环境

- CPU 原型：Windows Server，Intel 4 个物理核、8 个逻辑核（与其他任务共享），Python 3.12.10，torch 2.6.0+cpu；训练用 8 线程，decode 计时用 1 线程（`meta.json`）。
- GPU：AutoDL 上的 NVIDIA GeForce RTX 5090（sm_120，170 SM，32 GB GDDR7，`hw.jsonl` 报告 33.67 GB 可见），torch 2.8.0+cu128，CUDA 12.8，transformers 5.21.0（只用来对照 logits）。GPU 作业通过共享的 `flock` 串行执行，避免与其他实验互相干扰；进程绑定在 2 个 CPU 核上（`taskset -c 41,43`）。
- 实测硬件峰值（`hw.jsonl`）：2 GiB `copy_` 的有效带宽为 1522 GB/s（读和写各算一次）；fp16 GEMM 8192^3 达到 223.4 TFLOP/s。后文所有 roofline 预测都用这两个实测值构造 `mlsys_calc.GPU("RTX 5090 (measured)", ...)`，没有用规格书上的 1.79 TB/s。
- 数据：推测解码与正确性检查的 prompt 取自 WikiText-2 [9]（raw，test split）的 GPT-2 token 流，随机截取 64-token 的窗口；推测解码主实验用 100 个 prompt，每个生成 64 个新 token。
- 精度：正确性对照用 fp32（关闭 TF32），所有计时用 fp16。

3 方法与实现

这一节交代实现上的几个关键决定：权重怎么加载、KV cache 怎么设计才能配合 CUDA Graph、推测解码怎样避免频繁的 CPU-GPU 同步，以及接受率该怎么估计。

3.1 GPT-2 前向与权重加载

HF 的 GPT-2 用的是 `Conv1D`，权重形状为 (d_{in}, d_{out})。我们直接读 `model.safetensors`，转置成 `F.linear` 需要的 (d_{out}, d_{in}) 布局并 `contiguous()`；LM head 与 `wte` 共享权重；GELU 用 tanh 近似，也就是 HF 的 `gelu_new`。Listing 1 是一层的核心代码。三种 attention 路径共用同一套线性层，区别只在 K/V 从哪里来：

Listing 1: `gpu/gpt2_fast.py`：一层的 forward，三种 KV 路径

```
qkv = F.linear(_ln(x, l1w, l1b), qkvw, qkvb).view(B, t, 3, H, hd)
q, k, v = (z.transpose(1, 2) for z in qkv.unbind(2))
if cache is None: # (a) no cache: recompute the whole prefix
   y = F.scaled_dot_product_attention(q, k, v, is_causal=True)
elif pos is None: # (b) eager KV: write slice, attend to prefix
   cache.k[i, :, :, s:s + t] = k; cache.v[i, :, :, s:s + t] = v
   y = F.scaled_dot_product_attention(q, cache.k[i, :, :, :s+t], cache.v[i, :, :, :
       s+t], attn_mask=mask)
else: # (c) static, CUDA-graph-safe: position is a device tensor
   cache.k[i].index_copy_(2, positions, k); cache.v[i].index_copy_(2, positions, v)
   y = sdpa_or_bmm(q, cache.k[i, :, :, :t_attn], cache.v[i, :, :, :t_attn], mask)
x = x + F.linear(y.transpose(1, 2).reshape(B, t, C), ow, ob)
x = x + F.linear(F.gelu(F.linear(_ln(x, l2w, l2b), fcw, fcb), approximate="tanh"),
    prw, prb)
```

3.2 静态 KV cache 与 CUDA Graph

KV cache 预分配成两个形状为 $(L, B, H, T_{\max}, d_h)$ 的张量，用 `length` 标出有效前缀的长度。这样设计的好处是，推测解码拒绝 draft 时只需把 `length` 调小即可回滚，不用搬动数据。

CUDA Graph 把一串 kernel 录下来，之后一次性重放（replay），省掉逐个启动的 CPU 开销。它的要求是每次 replay 时张量的地址和形状都不变。所以静态路径把「当前位置」做成一个 0 维的 device 张量 `pos`，写 cache 用 `index_copy_`，attention 在固定长度 t_{attn} 的前缀上做，再用 `arange(t_attn) <= positions` 生成的 mask 屏蔽未来的位置。这里有一个顺序上的坑：graph 的 warm-up 和 capture 会在位置 $[0, t)$ 写入垃圾 K/V，所以必须先 *capture*、再 *prefill*。prefill 会覆盖这些位置，而 mask 保证无效位置永远不会被读到。

Listing 2: `GraphStep`：一次 capture，之后每步只做两次小拷贝 + replay

```
self.idx = torch.zeros(B, t, dtype=torch.long, device=dev)
self.pos = torch.zeros((), dtype=torch.long, device=dev)
... warm-up on a side stream ...
with torch.cuda.graph(self.graph):
   self.out = model.forward(self.idx, cache, self.pos, self.t_attn)
def __call__(self, idx, pos):
   self.idx.copy_(idx); self.pos.fill_(pos); self.graph.replay(); return self.out
```

静态路径有两种 attention 实现。一种是 `sdpa`，即 PyTorch 的 `scaled_dot_product_attention` 加 bool mask；另一种是 `bmm`，显式地做 $QK^\top$、softmax、PV 三个算子。后者是我在实验中发现前者在 batch 1 decode 时随上下文变慢之后才加的，详见第 4.3 节。

3.3 GPU 推测解码

`SpecEngine` 针对一对 (target, draft) 和固定的 γ，预先 capture 四个 graph：target 的 t=1（作基线）与 t=γ+1（作验证），draft 的 t=1 与 t=2。draft 需要 t=2，是因为它每轮最多会落下 2 个 token：全部接受时，它既没见过 x_γ，也没见过 bonus token。

每一轮的流程如下。draft 先自回归地提议 γ 个 token，采样在 GPU 上进行，不与 CPU 同步；然后 target 对 $[\text{seq}_{-1}, x_1..x_\gamma]$ 做一次 forward；接受-拒绝也全部在 GPU 上算完，最后用一次 `tolist()` 同步取回 $(n_{acc}, \text{next}, \text{overlap})$。为了让回滚始终正确，代码维护一个缓存不变量：每轮开始时，target cache 恰好存着 `seq[:-1]`，draft cache 存着 `seq` 的一个前缀（缺 1 或 2 个 token）。

接受-拒绝的实现如下。关键是用 `cumprod` 找出第一次拒绝的位置，从而避免在 CPU 上逐个判断：

Listing 3: `accept_reject_torch`：无主机同步的接受-拒绝

```
ratio = p[r, drafts] / q[r, drafts] # p: (g+1,V) target, q: (g,V) draft
u = torch.rand(g, device=p.device, generator=gen)
n_acc = torch.cumprod((u < ratio).long(), 0).sum() # index of first rejection
resid = torch.cat([(p[:g] - q).clamp_min(0), p[g:]])[n_acc] # norm(max(p-q,0)) or
    bonus p_g
nxt = torch.multinomial(resid / resid.sum(), 1, generator=gen)[0]
```

贪心版本把接受条件换成 $x_i = \arg\max p_i$，在第一个不一致的位置取 target 的 argmax。

3.4 接受率 α 的定义

接受率看似简单，却最容易估错。一轮里第一次拒绝之后的 draft 根本没有被检验过，如果把它们也算进分母，接受率就会被系统性地低估。所以这里沿用 CPU 版 README 的结论，用 $\hat{\alpha} = \frac{\#\text{accepted}}{\#\text{accepted}+\#\text{rejections}}$（截断几何分布的极大似然估计），而不是 #accepted/#proposed。采样模式下还记录了 $\mathbb{E}[\sum_x \min(p,q)] = 1 - \text{TV}(p,q)$，这是 Leviathan 定义的 α，它应当与 $\hat{\alpha}$ 一致。

3.5 理论模型

设每次 draft forward 的代价是 target 单 token forward 的 c 倍，target 一次验证 γ+1 个 token 的代价是单 token 的 v 倍。那么每轮期望产出的 token 数和加速比分别是

$$\mathbb{E}[\text{tokens/round}] = \frac{1-\alpha^{\gamma+1}}{1-\alpha}, \qquad S(\gamma) = \frac{\mathbb{E}[\text{tokens/round}]}{\gamma c + v}. \tag{1}$$

v=1 时就是 `mlsys_calc.speculative_speedup`。decode 一步的 roofline 下界来自 `mlsys_calc.decode_step_time`，取计算时间和访存时间中较大的一个：

$$t_{\text{step}} \geq \max\left(\frac{2NB + 4LdTB}{P_{\text{peak}}}, \frac{2N + 2\cdot 2LdTB}{BW}\right), \tag{2}$$

其中 N 为参数量（含共享的词嵌入），B 为 batch，T 为上下文长度；fp16 每个元素 2 字节，KV 每个 token 每层有 $2d$ 个元素。

4 实验结果

结果按这样的顺序展开：先用 CPU 原型确认算法正确、建立直觉，再在 GPU 上验证正确性，然后依次看 KV cache 的延迟、显存，最后是推测解码的接受率、加速和失效条件。

4.1 CPU 原型：KV cache 与推测解码（已有部分）

CPU 原型的 target 有 796,928 个参数（4 层，宽 128），draft 有 29,712 个（1 层，宽 48），在一个由 64 个随机「单词」组成的稀疏 bigram 合成语法上训练（`models.json`）。

先看 KV cache。图 1 左图是单步延迟。不用 cache 时，第 t 步要对 t 个 token 做完整的 forward，延迟从位置 16 的 2.27 ms 线性涨到位置 1008 的 59.0 ms；用 KV cache 后每步只算 1 个 token，延迟从 1.42 ms 缓慢涨到 2.25 ms，涨的那部分来自 attention 的 $O(t)$ 计算。右图是总耗时。T 从 512 增加到 1024 时，无 cache 的耗时涨了 4.27 倍，符合 $\sim T^2$；KV 的耗时只涨了 2.15 倍，符合 $\sim T$。T=1024 时，KV 的生成速度是 499.2 tok/s，是无 cache（38.9 tok/s）的 12.82 倍（`kv_timing.csv`）。

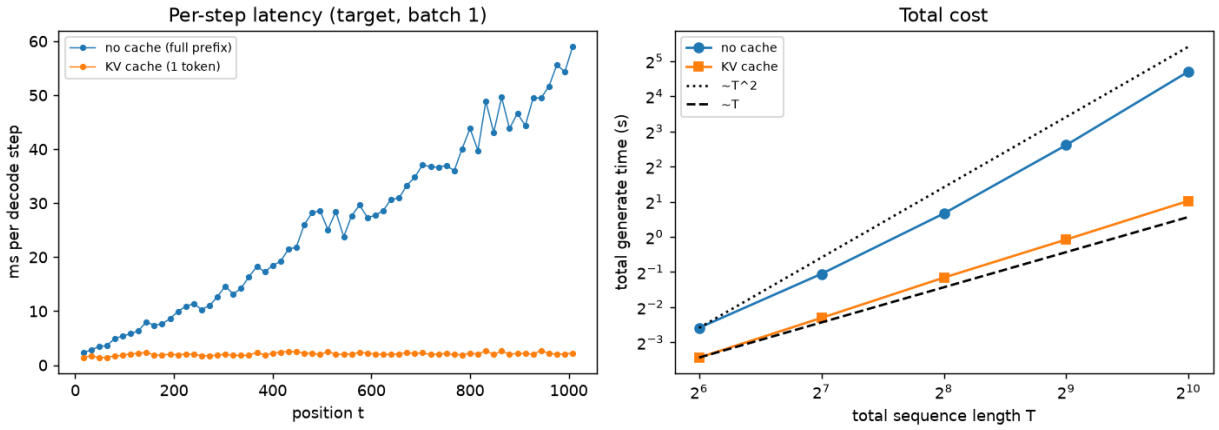


图 1: CPU：单步延迟（左）与总生成时间（右，log-log，虚线为 T 与 T^2 的斜率）。

再看推测解码。图 2 左图中，实测的「每次 target 调用产出的 token 数」与 $(1-\alpha^{\gamma+1})/(1-\alpha)$ 相差在 4% 以内。贪心 γ=8 时是 8.24 对 8.56，差距来自每个 prompt 的最后一轮，那里的 γ 被截短了。但右图的实测加速只有 0.98–1.99 倍，明显低于「验证免费」的预测。原因有两个：CPU 上验证 γ+1 个 token 的 forward 要花单 token 的 1.41–1.61 倍（`speculative.csv` 的 `verify_cost_vs_1tok`）；而 draft 的相对代价 c=0.256 被 Python 调用开销抬高了，参数量之比其实只有 0.04。

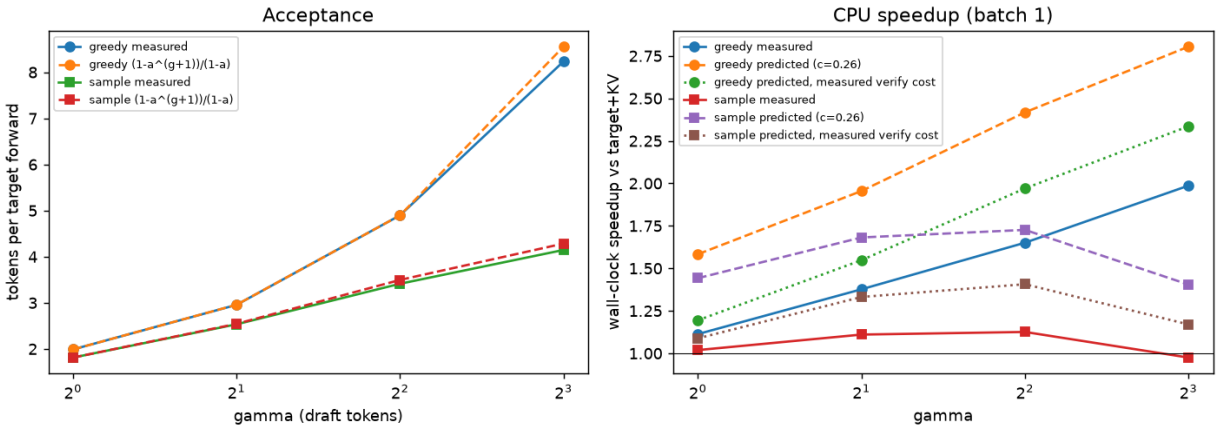


图 2: CPU：每次 target forward 产出的 token 数（左）与墙钟时间加速（右）。

结论：CPU 原型

CPU 上 KV cache 的收益来自省掉 $O(T)$ 倍的线性层计算（T=1024 时快 12.82 倍）。推测解码的 token 数公式很准，但加速被两件事吃掉了：验证不免费（$v \approx$ 1.4–1.6），以及 c 被调用开销抬

高。这两点到了 GPU 上会怎样变化，就是 GPU 实验要回答的问题。

4.2 GPU 正确性：与 HF 对齐，贪心输出一致

在测速度之前，要先确认自己写的 forward 算的是同一个模型，否则后面所有的加速都没有意义。表 2 与图 3 给出 fp32（关闭 TF32）下，我们的 forward 与 HF `GPT2LMHeadModel`（eager attention）在 2 条 128 token 的 WikiText 输入上的 logits 差。

gpt2-large / gpt2-xl 的最大绝对误差为 4.2×10^{-5} / 4.6×10^{-5}，满足 $\sim 10^{-4}$ 的要求。gpt2 与 gpt2-medium 的绝对误差更大（2.26×10^{-3} / 4.58×10^{-4}），但这是因为这两个模型的 logits 本身就很大，最大 |logit| 为 282.2 / 348.4。换成相对误差只有 8.0×10^{-6} / 1.3×10^{-6}，softmax 概率的最大差 $\leq 9.5 \times 10^{-6}$，argmax 一致率都是 1.0。KV cache 路径（64 token prefill 加 64 次单 token decode）与 HF 的差在同一量级。

表 2: fp32 logits 对照 HF transformers 5.21.0（`verify.jsonl`）

模型	无 cache 最大 $\lvert\Delta\rvert$	KV 最大 $\lvert\Delta\rvert$	平均 $\lvert\Delta\rvert$	最大 $\lvert\text{logit}\rvert$	相对误差	最大 $\lvert\Delta p\rvert$
gpt2	2.26e-3	1.48e-3	7.2e-5	282.2	8.0e-6	9.5e-6
gpt2-medium	4.58e-4	1.04e-3	4.1e-5	348.4	1.3e-6	5.6e-6
gpt2-large	4.20e-5	4.58e-5	1.5e-6	48.6	8.6e-7	2.8e-6
gpt2-xl	4.58e-5	3.81e-5	2.0e-6	50.9	9.0e-7	2.2e-6

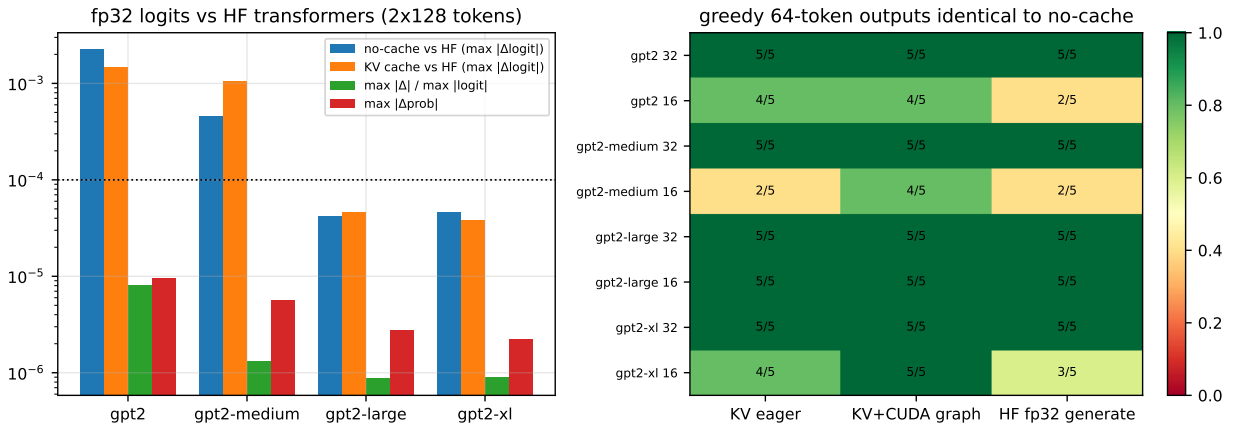


图 3: 左：fp32 logits 误差（log 轴，虚线为 10^{-4}）；右：5 个 prompt 各生成 64 个新 token，贪心输出与无 cache 版本完全相同的比例，按模型与精度分行。

logits 接近还不够，生成的序列也应当一致。fp32 下，4 个模型各 5 个 prompt，KV eager、KV 加 CUDA Graph、HF `generate` 三者生成的 64 个新 token，与无 cache 版本全部逐 token 相同（60/60）。

fp16 下则出现了分叉，例如 gpt2-medium 的 KV eager 只有 2/5 与无 cache 相同。这不是 bug。fp16 下不同形状的 GEMM（整段 prefill 对单 token）累加顺序不同，在 top-2 logit 几乎相等的位置就可能翻转 argmax，之后序列自然就分开了。推测解码实验里我们直接测了分叉点的 top-2 差（第 4.7 节），全部 ≤ 0.0078，正好是 fp16 在这个量级上的 1 个 ulp（最后一位的单位）。

结论：正确性

「贪心输出必须一致」只能在 fp32 下当作硬约束来检查（本实验 60/60 通过，推测解码 fp32 下 120/120 通过）；fp16 下应当改为「分叉只发生在 top-2 差 $\leq$ 1 ulp 的平局处」。

4.3 每 token 延迟与上下文长度、模型大小的关系

教科书上说 KV cache 能把每步的计算从 $O(T)$ 个 token 降到 1 个，所以应该总是更快。这一节检验这个说法在 batch 1 的 GPU 上是否成立，以及省下来的时间最后去了哪里。图 4 左图是 batch 1、fp16 下三种实现的每 token 延迟，有几个现象出乎直觉：

1. 短上下文时，KV eager 比无 cache 还慢。gpt2-xl 在上下文 16 时，无 cache 是 4.84 ms，KV eager 是 6.21 ms；直到上下文 128 两者才大致持平（6.71 对 6.16 ms）。原因是 eager 模式每层要多发 cache 写入、切片、mask 等小 kernel。batch 1 时 GPU 大部分时间都在等 CPU 发 kernel，也就是受启动开销限制（launch-bound），省下的 FLOP 根本不在关键路径上。

2. 无 cache 的代价要到长上下文才显现。gpt2-xl 在上下文 1024 时，无 cache 每 token 20.62 ms，KV eager 6.32 ms，KV 加 CUDA Graph（bmm attention）4.22 ms，比无 cache 快 4.9 倍。

3. CUDA Graph 消掉了启动开销。gpt2-xl 在上下文 16 时从 6.21 ms（eager）降到 3.74 ms（graph），gpt2 从 1.56 ms 降到 0.49 ms（`latency.jsonl`）。

4. 用 SDPA 加 bool mask 的 decode attention 随上下文明显变慢。gpt2-xl 从上下文 16 的 3.74 ms 涨到 1024 的 6.36 ms，而 roofline 预测只该涨 0.20 ms（KV 读取量 315 MB 除以 1.52 TB/s）。问题出在并行度上：batch 1 时 attention 只有 $B \times H = 25$ 个 (batch, head) 对，SDPA 的 memory-efficient kernel 按这些对来分 block，170 个 SM 大多闲着。我把它改成显式的 $QK^\top$、softmax、PV 之后（cuBLAS 会沿 T 维切分），gpt2-xl 在上下文 1024 时降到 4.22 ms，曲线几乎变平。这正是 Flash-Decoding [6]（沿 KV 维切分）要解决的问题。

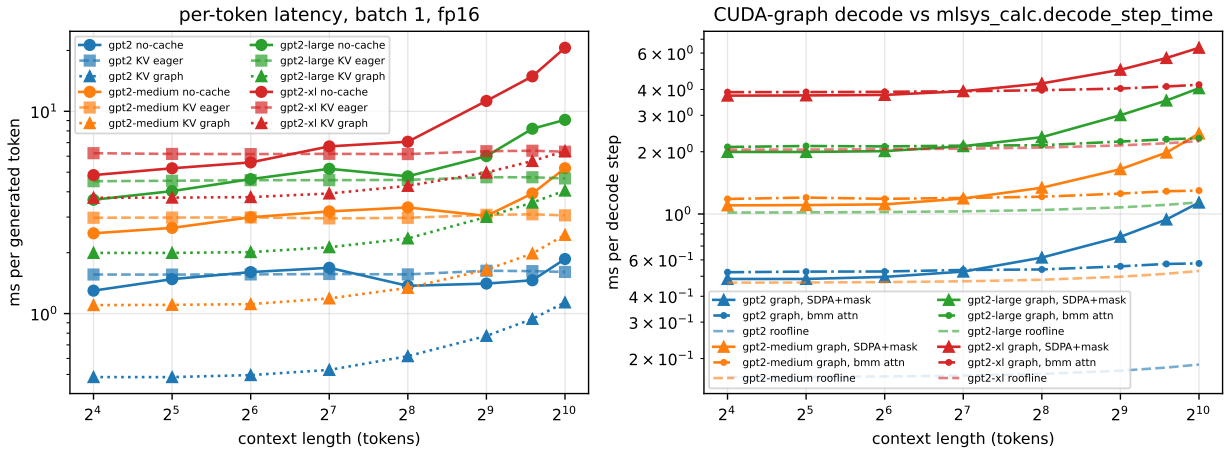


图 4: 左：三种实现的每 token 延迟（log-log）；右：CUDA Graph decode（SDPA+mask 与 bmm 两种 attention）对比 `mlsys_calc.decode_step_time` 的 roofline 下界（虚线）。

即使用上 CUDA Graph 和 bmm attention，延迟离 roofline 下界还有不小的距离。为了找出这段差距来自哪里，我换了一个角度，把延迟画成模型大小的函数。

图 5 把上下文 128 时的延迟画成权重字节数的函数。CUDA Graph 的延迟几乎是权重字节数的线性函数，但斜率约为「权重除以带宽」那条虚线的 2 倍：gpt2-xl 实测 3.92 ms，roofline 是 2.07 ms。如果差距只是带宽利用率不够，它应该与权重字节成正比；但图 6 把差值（实测减 roofline）对层数作图后，发现它与层数近似成正比。gpt2-large 与 gpt2-xl 每层多出 30.4–38.6 µs，相当于每层约 14 个 kernel、每个 2–3 µs 的固定开销。即使在 graph 里，kernel 启动、尾部 wave 和依赖等待也依然存在。加入 bmm attention 后，我在 4 个模型、3 个上下文长度（16 / 128 / 1024）上重新拟合，每层开销都落在 29.9–41.0 µs 之间（直线拟合的斜率为 40.0–42.9 µs/层，`decode_overhead.csv`），与上下文长度基本无关。

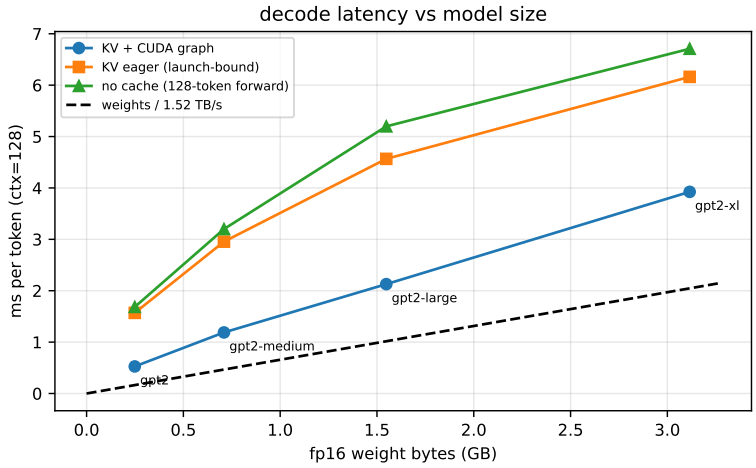


图 5: 上下文 128 时每 token 延迟与 fp16 权重字节数的关系；黑色虚线为「权重 / 实测带宽」。

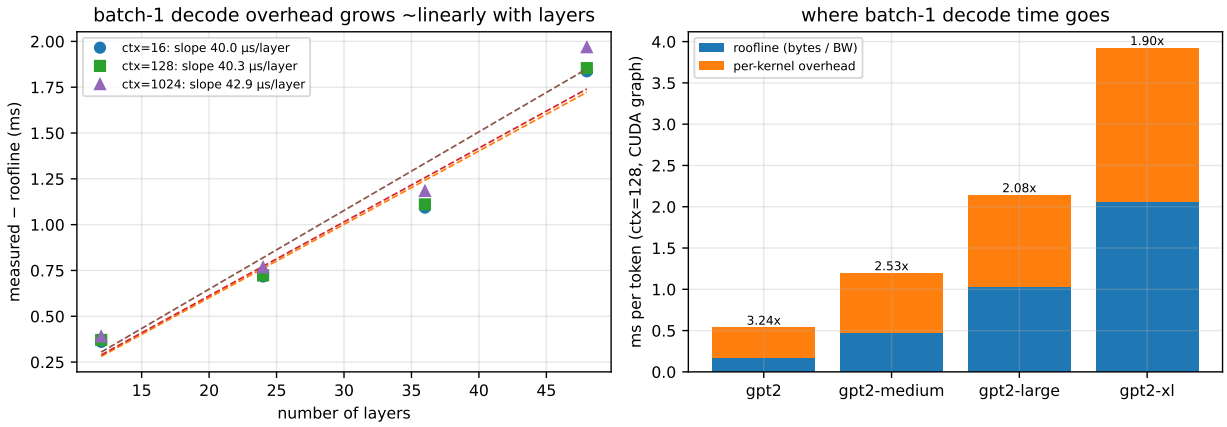


图 6: 左：batch 1 decode 的「实测 − roofline」随层数线性增长（三个上下文长度各拟合一条直线）；右：上下文 128 时把延迟拆成 roofline 部分与固定开销部分，柱顶标注实测/预测比。

结论：batch 1 decode 的时间去哪了

在 RTX 5090 上，gpt2-xl 的 batch 1 decode 只用到实测带宽的约一半（实测/roofline = 1.90），另一半是与层数成正比的 per-kernel 固定开销。模型越小，这部分占比越大（gpt2 是 3.24 倍）。这也是推测解码里小 draft 的 c 远大于参数量之比的原因。

4.4 Prefill 与 decode

上一节只看 decode。实际生成时还有一个 prefill 阶段，即一次性处理整个 prompt。这一节比较两者的代价，看看时间主要花在哪个阶段。

图 7 是生成 128 个新 token 时 prefill 与 decode 的时间拆分。prefill 一次处理整个 prompt，做的是对算力友好的大 GEMM。gpt2-xl 处理 512 个 prompt token 的 prefill 只要 12.80 ms，而之后 127 个 decode token 要 680.0 ms（CUDA Graph）。折算下来，decode 每个 token 的代价约是 prefill 中每个 token 的 214 倍。无 cache 版本的 decode 时间随 prompt 变长而增长（gpt2-xl 在 P=128 时 853.4 ms，P=512 时 1627.8 ms），KV 版本则基本不变（eager 下 782.8 / 796.5 ms）。

图中有一处反常：CUDA Graph 版本在 P=896 时（810.5 ms）反而比 eager（794.6 ms）慢。原因是 `generate_graph` 的 attention 固定在整个 $T_{\max} = 1024$ 的缓冲区上用 SDPA 加 mask，这把第 4.3 节 里 SDPA 的长上下文问题放大了。

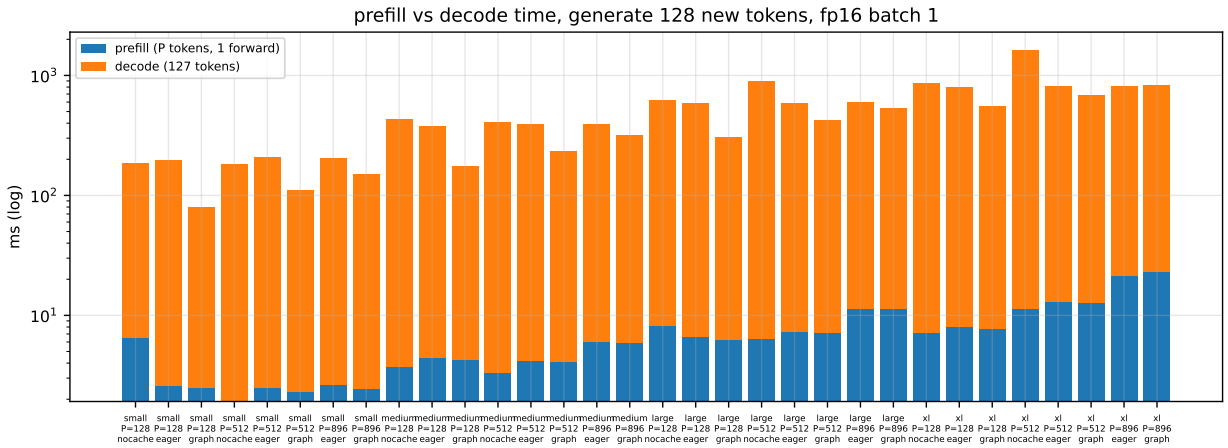


图 7: prefill（下）与 decode 127 个 token（上）的时间，log 轴；横轴依次为模型、prompt 长度和实现。

4.5 吞吐与 batch：从 memory-bound 到 compute-bound 的拐点

batch 1 时 GPU 的算力大部分闲着。最直接的补救办法是同时服务多个序列，让一次权重读取被多个 token 共享。这一节看加大 batch 能把吞吐推到多高，以及拐点在哪里。

图 8 是 CUDA Graph decode 的 tokens/s 随 batch 的变化（实线），虚线是 `mlsys_calc.decode_step_time` 用实测峰值（1522 GB/s、223.4 TFLOP/s）给出的上界。上下文 256 时，gpt2-xl 的步时间从 batch 1 的 4.27 ms 到 batch 16 的 5.27 ms 几乎不变，因为权重只读一次，被 16 个序列共享，所以吞吐近似线性地增长到 3,039 tok/s。此后 KV 的读取量随 batch 线性增长。batch 256 时 KV 达到 20.1 GB，而权重只有 3.1 GB；步时间为 20.0 ms，吞吐 12,801 tok/s，仍然受带宽限制。也就是说，在上下文 256 下，GPT-2 的 decode 永远到不了算力屋顶，因为强度被 KV 封顶了。

为了看到只考虑权重时的拐点，我又在上下文 32 下重复了 batch 扫描（`ctx32/batch.jsonl`），这时 KV 很小，强度近似等于 B。gpt2-xl 的步时间在 B=2–16 时几乎不变（4.14–4.28 ms），吞吐从 B=1 的 268.7 tok/s 涨到 B=16 的 3,739 tok/s（13.9 倍）。B=32 起步时间开始上升（5.44 ms）；从 B=16 到 256，batch 又翻了 16 倍，吞吐却只再涨 6.95 倍，到 25,999 tok/s。B=256 时强度为 142 FLOP/B，已经接近 ridge（147），实测 $2NB/t$ = 80.9 TFLOP/s，只是 GEMM 实测峰值的 36%。换句话说，实测拐点（$B \approx 16$–32）比 roofline 拐点（$B \approx 147$）早得多。原因是小 M 的 GEMM 达不到峰值，B 一过 16，步时间就开始随 B 增长。

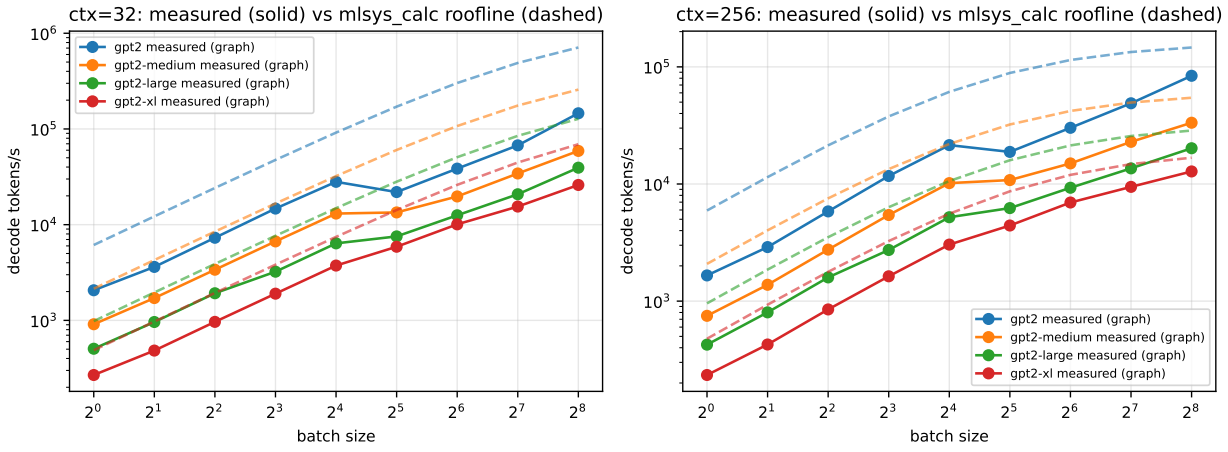


图 8: decode 吞吐随 batch 的变化（log-log），实线为实测（CUDA Graph），虚线为 roofline 预测。

图 9 换成每步时间来看同一组数据，并加入了 eager 模式作对比。小 batch 时 eager 比 graph 多出一段固定的启动开销；batch 变大后计算本身变长，这段开销被摊薄，两条线重合。

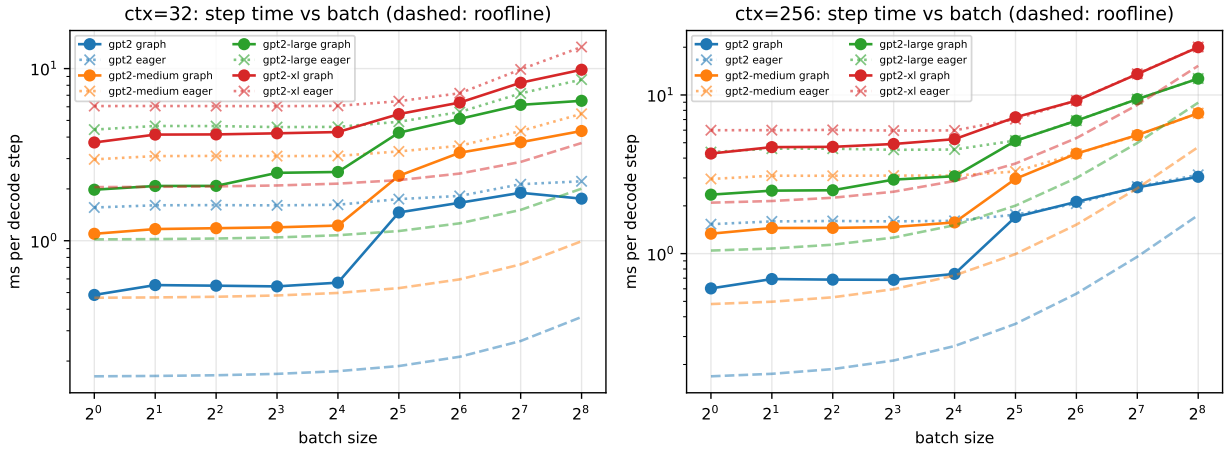


图 9: 每步时间与 batch 的关系：graph（实线）、eager（点线）、roofline（虚线）。小 batch 时 eager 与 graph 相差一段固定的启动开销，大 batch 时两者重合。

最后把所有配置放到同一张 roofline 图上，并直接看实测与预测之比，这可以告诉我们 roofline 公式在多大程度上能当作延迟预测来用。

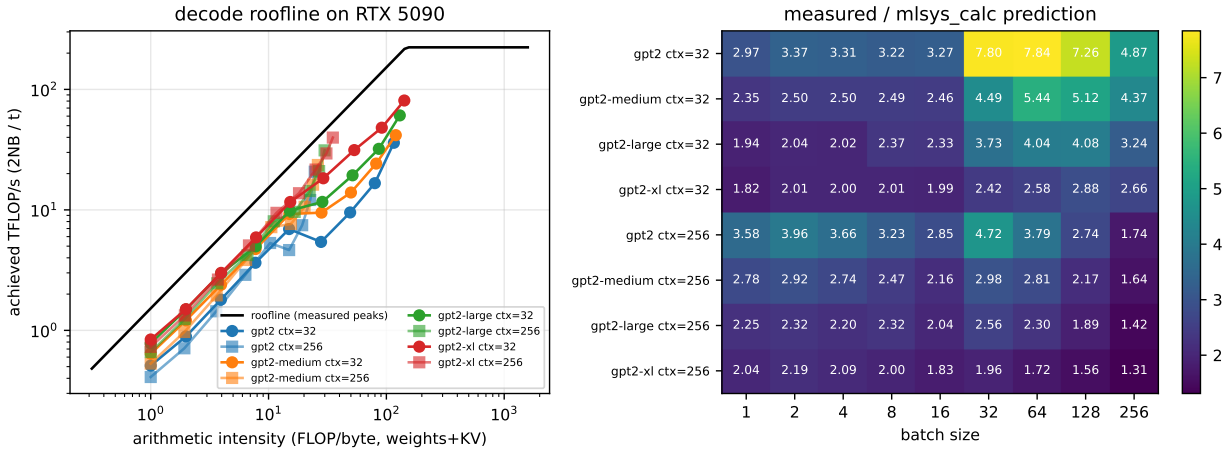


图 10: 左：roofline 图，横轴为 `decode_step_time` 给出的强度（FLOP / 权重 +KV 字节），纵轴为实测 $2NB/t$；右：实测与预测时间之比。

图 10 右图的比值是本节最重要的「模型对实测」结论。上下文 256 下，gpt2-large / gpt2-xl 的实测/预测比在 1.31–2.56 之间，并随 batch 增大而下降（xl 在 B=1 时为 2.04，B=256 时为 1.31），因为固定开销被摊薄了，KV 读取的占比上升。gpt2 / gpt2-medium 在 1.64–4.72 之间，小模型的 per-kernel 开销占比更大。上下文 32 下比值更高（xl 为 1.82–2.89，gpt2 为 2.97–7.84），因为这时预测时间更短，而实测在 $B \geq 32$ 后又被 GEMM 效率拖慢。所以 `decode_step_time` 在这台机器上是一个可靠的下界：大模型、大 batch 时偏差在 1.3–2 倍，小模型或短上下文时可以差到 3–8 倍，不能直接拿来当延迟预测。

4.6 KV cache 显存：实测与公式

KV cache 用显存换时间，而它占的显存往往比权重还多。这一节检验教科书上的 KV 显存公式是否与 PyTorch 实际分配的一致。

图 11 覆盖 4 个模型、$T \in \{128, 256, 512, 1024\}$、$B \in \{1, 8, 32, 64\}$，共 63 个组合（超过 16 GB 的跳过）。对每个组合，比较 `torch.cuda.memory_allocated` 在分配 `StaticKV` 前后的差，与公式 $2 \cdot L \cdot B \cdot H \cdot d_h \cdot T \cdot 2$ 字节（等于 `mlsys_calc.kv_cache_bytes`）。58/63 个组合逐字节相等。其余 5 个都是 batch 1 的小缓冲区，被 PyTorch 的 caching allocator 向上取整了，例如 gpt2-large T=128 的公式值为 23,592,960 B，实测 25,165,824 B = 24 MiB。最大的偏差是 gpt2 T=128, B=1，为 1.39 倍（4.72 MB 对 6.55 MB）。decode 一步的额外峰值（激活加 workspace）只有 1.1–9.6 MB，与 KV 相比可以忽略。

具体到 gpt2-xl，每个 token 的 KV 是 $2 \times 48 \times 1600 \times 2 = 307{,}200$ 字节（300 KiB）。T=1024, B=32 时共 10.07 GB，已经是权重（3.12 GB）的 3.2 倍。

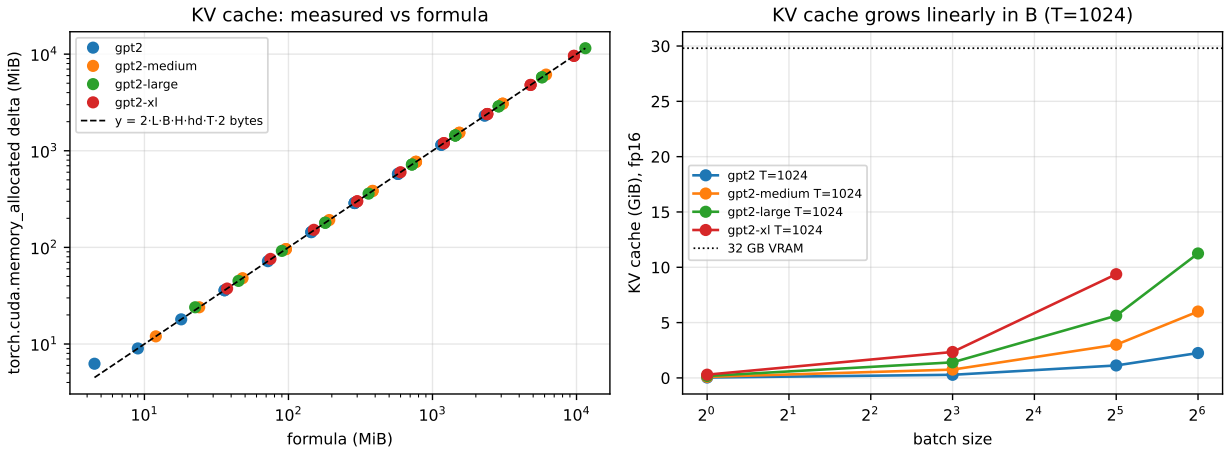


图 11: 左：实测 KV 分配对照公式（虚线为 $y = x$）；右：T=1024 时 KV 随 batch 线性增长（点线为 32 GB 显存）。

4.7 推测解码：接受率与每次 target 调用的 token 数

从这里开始转向推测解码。它的收益可以拆成两个因素：每次 target 调用能产出几个 token，以及每轮要付出多少代价。这一节先看前者，它由接受率决定。

主实验的设置是：draft 为 gpt2（124M），target 为 gpt2-large（774M）和 gpt2-xl（1.5B）；100 个 WikiText prompt，每个 64 token，每个生成 64 个新 token；$\gamma \in \{1, \ldots, 8\}$，温度 $T \in \{0, 0.7, 1.0\}$；全部使用 CUDA Graph 和 fp16。

图 12 显示 $\hat{\alpha}$ 基本不随 γ 变化。gpt2→gpt2-large 贪心约为 0.74–0.76，T=1 约为 0.66–0.68；gpt2→gpt2-xl 贪心为 0.70–0.72，T=1 为 0.63–0.65。采样模式下，$\hat{\alpha}$ 与 $\mathbb{E}[\sum_x \min(p, q)]$ 吻合得很好，例如 gpt2-xl γ=4, T=1 时是 0.6285 对 0.6423，gpt2-large γ=8, T=1 时是 0.6629 对 0.6644。相比之下，错误的定义 accepted/proposed 随 γ 单调下降，到 γ=8 只剩 0.21–0.34。用它代入公式，会严重低估加速。

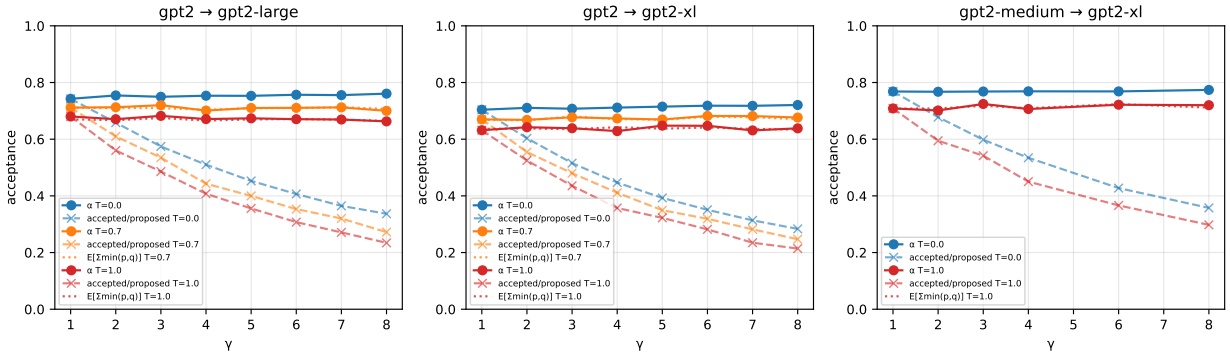


图 12: 接受率：正确的 $\hat{\alpha}$（实线）、错误的 accepted/proposed（叉号虚线）、$1 - \mathrm{TV}$ 的期望（点线，仅采样）。

有了 $\hat{\alpha}$，就可以检验每次调用的 token 数是否符合公式。图 13 中，采样模式下实测值与 $(1 - \hat{\alpha}^{\gamma+1})/(1 - \hat{\alpha})$ 几乎重合（gpt2-xl γ=8, T=1 时为 2.713 对 2.716）；贪心模式下 γ 较大时实测略低（gpt2-xl γ=8 时为 3.268 对 3.392）。

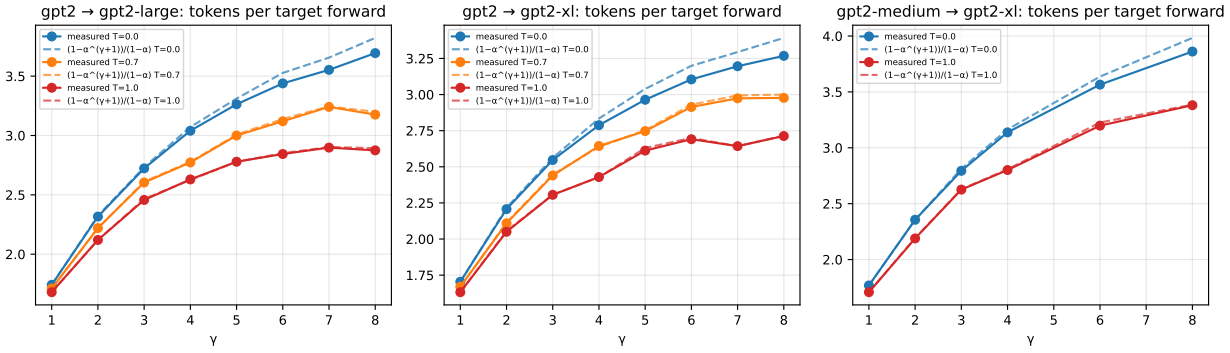


图 13: 每次 target forward 的 token 数：实测（实线）对照公式（虚线）。

贪心模式为什么会偏低？图 14 给出了原因：贪心时每轮接受数的分布比截断几何分布「两头更重」，0 个接受和全部接受的情况都偏多。以 gpt2-large γ=8 为例，全部接受的比例约为 0.15，几何分布预测的是约 0.11。这说明接受事件并不是独立同分布的：有些上下文里 draft 几乎总是猜对（套话、专有名词的后半段），有些几乎总是猜错。α 在不同上下文间的这种异质性使得 $\mathbb{E}[\alpha^k] \neq \bar{\alpha}^k$，而公式基于平均的 α，所以在 γ 较长时有偏。

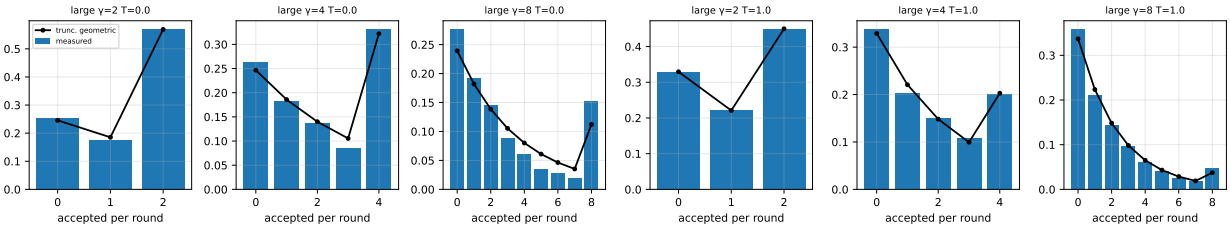


图 14: 每轮接受数的分布（gpt2→gpt2-large），对照用 $\hat{\alpha}$ 计算的截断几何分布（黑线）。

推测解码号称无损，贪心模式下这一点可以直接检验。fp32 下（gpt2→gpt2-xl，20 个 prompt 各 64 token，$\gamma \in \{1, 4, 8\}$，eager 与 CUDA Graph 两种实现），推测解码的输出与 target 单独贪心解码 120/120 完全一致（`spec_exact.jsonl`）。fp16 主实验中，gpt2-large 有 94/100、gpt2-xl 有 91/100 一致。不一致的 prompt 在所有 γ 下都是同一批，分叉点 target 的 top-2 logit 差最大只有 0.0078125，

多数为 0，也就是 fp16 下的平局。

4.8 推测解码：墙钟时间加速

token 数只是收益的一半，另一半是代价：draft 每步要多少时间（c），target 一次验证多个 token 要多少时间（v）。这一节把两者结合起来，看实际的墙钟加速。

表 3 摘录了主要结果，完整数据见 `spec_summary.csv`。基线是同一套 CUDA Graph 下 target 单独 decode 的速度：gpt2-large 为 429.3 tok/s，gpt2-xl 为 223.9 tok/s（贪心）。实测 $c = t_{\text{draft}}/t_{\text{target}}$ 对 gpt2-large 为 0.253，对 gpt2-xl 为 0.138（draft 单步 0.56 ms）。参数量之比分别只有 0.16 和 0.08，c 同样被 per-kernel 开销抬高了。验证代价 v 是 GPU 上推测解码成立的根本原因：gpt2-xl 在 γ=1–8 时 v 只有 1.10–1.12，也就是验证 9 个 token 只比验证 1 个贵 12%。gpt2-large 在 $\gamma \le 3$ 时 v 为 1.07，$\gamma \ge 4$ 时跳到 1.24。

表 3: 推测解码主要结果（100 个 prompt，每个 64 token，fp16，CUDA Graph；`spec_summary.csv`）

target	T	γ	$\hat{\alpha}$	tok/调用	公式	c	v	预测 $E/(\gamma c+v)$	实测加速
gpt2-large	0	3	0.750	2.723	2.733	0.253	1.070	1.494	**1.348**
gpt2-large	1.0	3	0.682	2.457	2.465	0.253	1.070	1.348	1.177
gpt2-large	1.0	5	0.674	2.779	2.778	0.253	1.242	1.108	0.986
gpt2-large	1.0	8	0.663	2.875	2.893	0.253	1.247	0.884	0.794
gpt2-xl	0	3	0.708	2.547	2.562	0.137	1.106	1.688	**1.631**
gpt2-xl	0	4	0.712	2.788	2.834	0.138	1.115	1.701	1.629
gpt2-xl	0.7	3	0.677	2.440	2.447	0.137	1.106	1.612	1.528
gpt2-xl	1.0	3	0.639	2.306	2.307	0.137	1.106	1.520	1.454
gpt2-xl	1.0	8	0.638	2.713	2.716	0.138	1.118	1.224	1.165

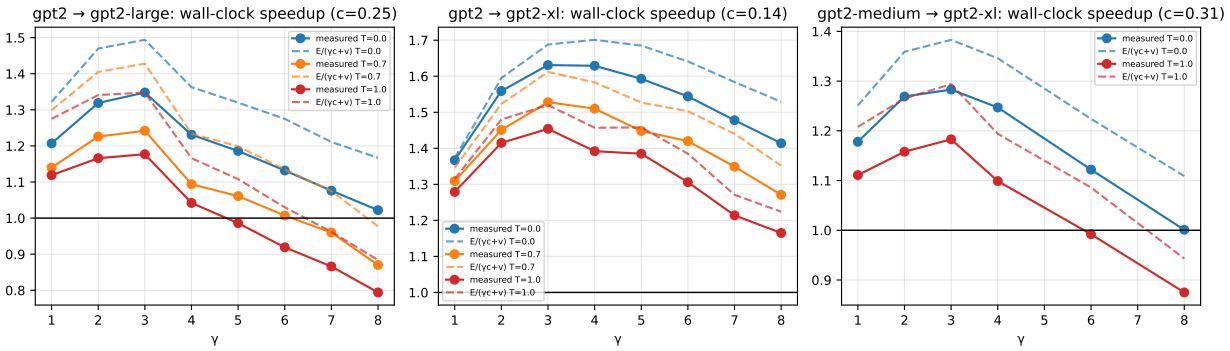


图 15: 墙钟加速随 γ 的变化：实测（实线）对照用实测 c、v、$\hat{\alpha}$ 代入式 (1) 的预测（虚线）。

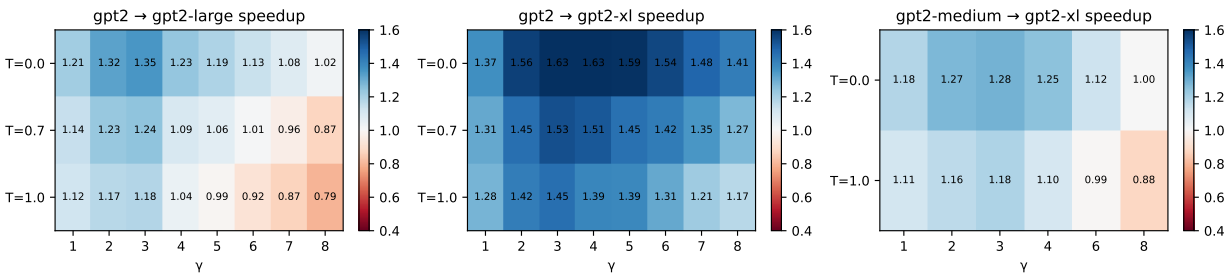


图 16: 加速热力图（行：温度，列：γ；蓝色 > 1，红色 < 1）。

从图 15 and 16 可以读出以下规律。

- 最优的 γ 很小。两个 target 的最优点都在 γ=3–4。因为 $\alpha \approx 0.64$–0.76 时 α^γ 衰减很快，多提议的 token 很少被接受，但每一个都要付出 c 的代价。
- 加速会变成减速。gpt2-large（c=0.253）在 T=1、$\gamma \geq 5$ 时低于 1（0.986，γ=8 时为 0.794）；T=0.7 时在 $\gamma \geq 7$ 后低于 1。gpt2-xl 的 c 只有 0.138，γ=8, T=1 时仍有 1.165 倍。target 越大，c 越小，能承受的 γ 就越大。
- 温度越高，加速越差。温度升高使 p 与 q 的 TV 距离变大，α 下降约 0.07–0.08，所有 γ 下的加速都随之下降。
- 公式偏乐观 0–13%。60 个配置的实测/预测比为 0.87–1.00（gpt2-xl γ=3, T=0 时为 1.631 / 1.688 = 0.966）。剩下的差距来自每轮一次的 GPU→CPU 同步、Python 端拼接 token 列表和 H2D 拷贝；式 (1) 也没有计入 draft 补喂 2 个 token 那一步比 1 个 token 稍贵。

一个自然的想法是换一个更大、更准的 draft。补充实验用 gpt2-medium（355M）作 gpt2-xl 的 draft（60 个 prompt，$T \in \{0, 1\}$），结果说明更大的 draft 不一定更好。贪心 $\hat{\alpha}$ 从 0.71 升到 0.77，每次调用的 token 数也更多（γ=3 时 2.794 对 2.547），但 c 从 0.138 涨到 0.308。于是最好的实测加速只有 1.283（γ=3，贪心），低于 gpt2 draft 的 1.631；T=1 时 γ=6 已降到 0.992，γ=8 为 0.875。用式 (1) 来看，α 从 0.71 提到 0.77 带来的收益，抵消不了 γc 翻倍的代价。这组实验的 fp16 贪心一致性为 54/60，分叉点的 top-2 差同样 ≤ 0.0078。

4.9 采样版推测解码不改变 target 分布

贪心模式的无损性可以逐 token 比较，采样模式就只能做统计检验了。这一节在真实模型上检验：推测解码采样出来的 token，分布是否真的等于 target 的分布。

实验设置如下：draft 为 gpt2，target 为 gpt2-large，fp32；固定一个 64-token 的 WikiText prompt，γ=3，每次生成 3 个 token；推测解码与 target 单独采样各重复 4000 次。对第一个 token，可以算出精确的 target 分布 p 与 draft 分布 q。把期望频数 ≥ 5 的 token 各自单独成箱，其余合并成一箱，然后做卡方检验（表 4）。

表 4: 分布检验（`dist.jsonl`，每组 4000 次）

温度	TV(p, q)	推测 vs p	推测 vs q	基线 vs p	token 2 双样本	token 3 双样本
$T = 1.0$	0.269	$p = 0.657$	$p = 8.9 \times 10^{-168}$	$p = 0.096$	$p = 0.307$	$p = 0.499$
$T = 0.7$	0.101	$p = 0.182$	$p = 3.4 \times 10^{-124}$	$p = 0.752$	$p = 0.381$	$p = 0.034$

推测解码的第一个 token 与精确的 p 没有显著差异（p 值 0.657 / 0.182），却以 10^{-124} 以下的 p 值被拒绝「来自 draft 分布 q」。后者很重要，它说明这个检验确实有足够的功效区分 p 和 q，前面的「无显著差异」不是因为样本太少。第 2、3 个 token 没有闭式分布，只能用推测解码与 target 单独采样做双样本卡方检验。6 个双样本检验中最小的 p 值是 0.034（T=0.7 的第 3 个 token），在 6 次检验的 Bonferroni 阈值 $0.05/6 = 0.0083$ 下并不显著，与「6 次里碰巧出现一次 $p < 0.05$」的偶然性一致。图 17 直观地对比了 top-15 token 的频率。顺带一提，全词表上的经验 TV 不是好指标：50,257 个 token 的长尾使 4000 次采样的 TV 噪声底就有 0.16，基线对精确 p 也是 0.165。

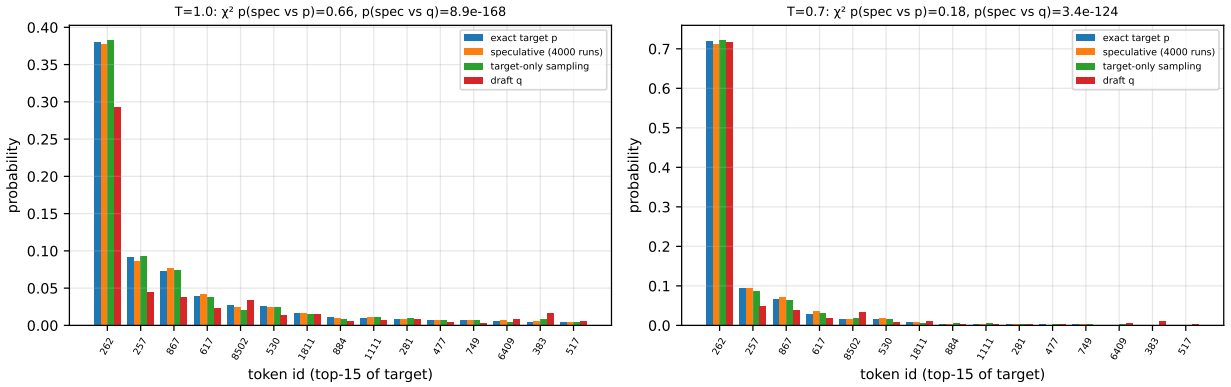


图 17: 第一个生成 token 的分布（top-15）：精确 target p、推测解码频率、target 单独采样频率、draft q。

4.10 推测解码在大 batch 下为什么失效

前面的加速都是 batch 1 的结果。推测解码的前提是「验证 γ+1 个 token 几乎和验证 1 个一样贵」，而这个前提只在 decode 受带宽限制时成立。batch 变大以后它还成立吗？

图 18 在 batch B 下测量 target 处理 γ+1 个 token 的时间与单 token 时间之比 $v(B)$，以及 draft 的 $c(B)$（上下文 128）。gpt2-xl 在 $B \le 8$ 时，验证 5 个 token 只贵 1.7–25%；到 B=128 时，验证 9 个 token 是单 token 的 2.40 倍（27.53 对 11.47 ms）。draft 的 c 在 $B \ge 32$ 时也从 0.14 涨到 0.20–0.25，因为 draft 自己也开始受 KV 读取和 GEMM 规模的影响。

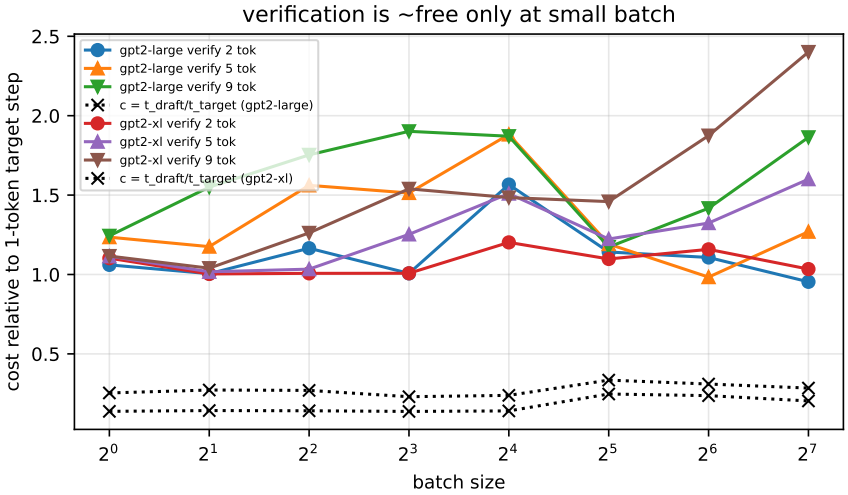


图 18: 相对单 token target step 的代价：验证 2/5/9 个 token 的 $v(B)$，以及 draft 的 $c(B)$（黑色点线）。

把实测的 $c(B)$、$v(B)$ 和主实验的 $\hat{\alpha}$ 代入式 (1)，得到图 19（`spec_batch_model.csv`）。gpt2-xl、T=1、γ=4 的预测加速从 B=2 的 1.528 降到 B=128 的 1.005；γ=8 在 $B \ge 32$ 时就低于 1（B=128 时为 0.674）。gpt2-large 在 B=16 时有一个低谷（γ=4, T=1 时为 0.926），对应 $v(16) = 1.88$：在这一点上，$B(\gamma{+}1) = 80$ 行的 GEMM 正好跨过了 cuBLAS 的 tile 边界。需要强调，这是用实测分量拼出来的预测，不是端到端测量。我们的 `SpecEngine` 只实现了 batch 1，因为不同序列的接受长度不同，需要管理参差不齐（ragged）的 KV。

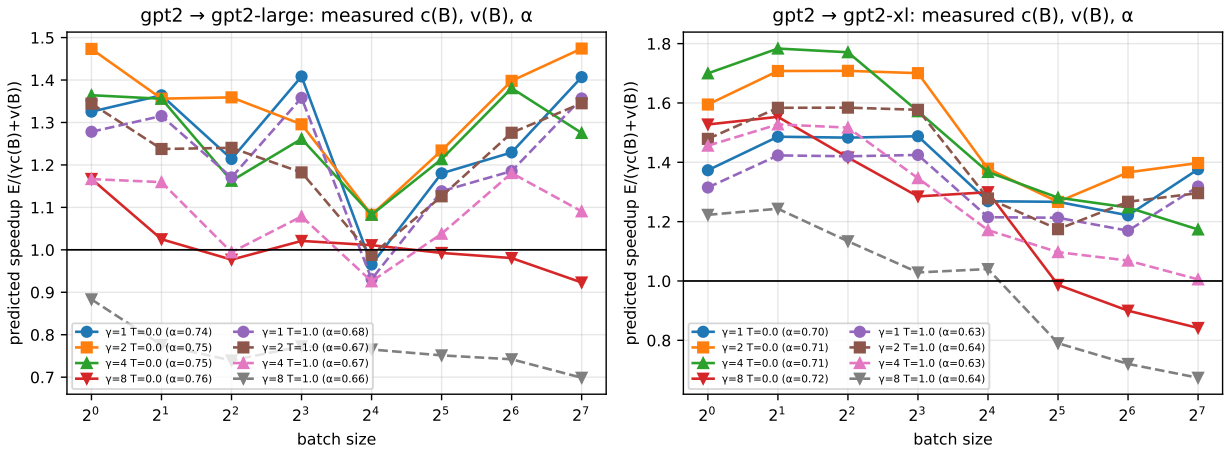


图 19: 用实测 $c(B)$、$v(B)$ 和 $\hat{\alpha}$ 预测的推测解码加速随 batch 的变化；黑线为 1。

结论：推测解码什么时候不划算

数据里出现了三个独立的失效方向：γ 太大（gpt2-large T=1、$\gamma \geq 5$ 时实测 < 1）；α 太低（温度从 0 升到 1，α 降约 0.07，所有配置的加速都下降）；batch 太大（gpt2-xl γ=8 在 $B \geq 32$ 时预测 < 1），因为 decode 逐渐脱离带宽受限，验证不再免费。

5　与理论模型的对照

这一节把前面用到的几个公式推导出来，并汇总模型与实测的差距，方便读者在新的模型或硬件上做估算。

5.1　decode 的 roofline

一次 decode 要读全部权重 $2N$ 字节（fp16）和全部 KV $2 \cdot 2LdTB$ 字节，做 $2NB + 4LdTB$ 次 FLOP。因此强度为

$$I(B) = \frac{2NB + 4LdTB}{2N + 4LdTB} \xrightarrow{T \to 0} B, \qquad I(B) \xrightarrow{B \to \infty} 1.$$

不计 KV 时，拐点在 $B^\star = P_{\text{peak}}/BW \approx 147$（实测峰值之比）；计入 KV 后，$B \to \infty$ 时强度趋于 1 FLOP/B，永远到不了 ridge。GPT-2 没有 GQA，每个 token 的 KV 是 $4Ld$ 字节，gpt2-xl 为 300 KiB。这相当于每 10,000 个 token 的上下文就是一份 3 GB 的权重，所以上下文 256 下 batch 一增大，KV 很快就占了主导。

5.2　推测解码

设每个 draft token 以独立的概率 α 被接受，那么第一次拒绝之前接受的个数 K 服从截断几何分布：$P(K{=}k) = (1-\alpha)\alpha^k$ $(k < \gamma)$，$P(K{=}\gamma) = \alpha^\gamma$。每轮产出 $K{+}1$ 个 token，所以

$$\mathbb{E}[K+1] = \sum_{k=0}^{\gamma} \alpha^k = \frac{1-\alpha^{\gamma+1}}{1-\alpha}.$$

α 的极大似然估计：每轮贡献「K 次成功，以及（若 $K < \gamma$）1 次失败」，似然为 $\prod \alpha^K (1-\alpha)^{\mathbb{1}[K<\gamma]}$，解得 $\hat{\alpha} = \sum K / (\sum K + \#\text{rejections})$，正是我们用的定义。每一轮的耗时是 $\gamma c + v$（以 target 单步为单位），合起来就是式 (1)。最优 γ 满足 $\partial S/\partial\gamma = 0$。以 gpt2-xl 贪心（$\alpha$=0.71，$c$=0.138，$v$≈1.11）代入，$S(\gamma)$ 在 γ=3, 4 取得最大值 1.69–1.70；实测的最优点也在 γ=3–4（1.631 / 1.629）。

5.3 分布无偏的证明

对任意 token x，输出它的概率由「被接受」和「被拒绝后从残差中采到」两部分组成：

$$P(\text{输出}=x) = q(x)\min\Big(1,\frac{p(x)}{q(x)}\Big) + \Big(1-\sum_y \min(p(y),q(y))\Big)\frac{\max(p(x)-q(x),0)}{\sum_y \max(p(y)-q(y),0)}$$
$$= \min(p(x),q(x)) + (p(x)-q(x))^+ = p(x),$$

第二个等号用到了 $\sum_y (p-q)^+ = 1-\sum_y \min(p,q)$。贪心版是 $T\to 0$ 的极限：p、q 都退化成 one-hot，当且仅当 draft 的 argmax 等于 target 的 argmax 时接受。

5.4 模型与实测的差距汇总

表 5: 模型与实测的对照

量	模型	实测	差距来源
gpt2-xl decode，B=1，ctx 128	2.07 ms	3.92 ms	每层约 39 µs 的 kernel 固定开销
gpt2-xl decode，B=256，ctx 256	15.27 ms	20.00 ms	attention kernel 读 KV 达不到峰值带宽（比值 1.31）
KV 字节（58/63 组合）	公式	逐字节相等	其余 5 个为 allocator 取整
tokens/调用（采样）	$(1-\alpha^{\gamma+1})/(1-\alpha)$	相差 $<1\%$	—
tokens/调用（贪心，γ=8）	3.392	3.268	α 在不同上下文间异质
加速（gpt2-xl γ=3，贪心）	1.688	1.631	每轮同步与 Python 开销

6 面试专题

6.1 20 秒电梯陈述

我在 RTX 5090 上用真实 GPT-2 权重从零实现了 KV cache、CUDA Graph decode 和推测解码。fp32 下与 HF 的 logits 差 $\le 4.6\times10^{-5}$（大模型），贪心输出 bit-exact。KV cache 在长上下文下把 gpt2-xl 每 token 从 20.6 ms 降到 4.2 ms；但 batch 1 decode 只用到一半带宽，另一半是 per-kernel 开销。推测解码 gpt2→gpt2-xl 贪心实测 1.63 倍，与 $(1-\alpha^{\gamma+1})/(1-\alpha)$ 加上实测 c=0.138、v=1.11 的预测相差 4%；我还用数据展示了 γ 过大、温度升高、batch 变大三种失效方式。

6.2 STAR 故事

情境（S）：推测解码的加速取决于 draft 的相对代价 c。按参数量算，gpt2/gpt2-xl 只有 0.08，理论加速应该很高。任务（T）：解释为什么实测 c 是 0.138，以及 decode 为什么只跑到 roofline 的一半。行动（A）：在 4 个模型上测 batch 1 graph decode，把「实测减 roofline」对层数作图，得到每层约 30–41 µs 的近似常数。之后又发现 SDPA 加 mask 在长上下文下变慢，于是把 decode attention 换成沿 T 切分的 bmm，gpt2-xl 在上下文 1024 时从 6.36 ms 降到 4.22 ms。结果（R）：用实测的 c、v 代入公式后，60 个配置的实测/预测加速比在 0.87–1.00 之间，并准确预测了 gpt2-large 在 $\gamma\ge5$、T=1 时会转为减速。

面试追问：为什么 decode 是 memory-bound，而 prefill 不是？

因为两者的算术强度差了几个数量级。decode 每步对每个权重只做 B 次乘加，强度约 B FLOP/B，而 5090 的 ridge 约 147 FLOP/B，batch 1 远低于它；prefill 一次处理 P 个 token，强度约为 P。实测 gpt2-xl prefill 512 个 token 只要 12.80 ms，而 decode 1 个 token 就要约 4–5 ms。

常见错答：「因为 attention 是 $O(T^2)$。」decode 每步的 attention 是 $O(T)$，batch 1 时主导时间的是读权重。

面试追问：KV cache 一定更快吗？

不一定。实测 gpt2-xl 在上下文 16 时，KV eager（6.21 ms）比无 cache（4.84 ms）还慢。因为 batch 1 时 GPU 在等 CPU 发 kernel，KV 路径多出来的小 kernel 反而拉长了关键路径。上下文 $\geq$128 时两者才持平，到 1024 时 KV 快 3.3 倍（eager）或 4.9 倍（graph 加 bmm）。

常见错答：「KV cache 永远带来 T 倍加速。」这只在计算或访存真正成为瓶颈时才成立。

面试追问：接受率 α 怎么估计？

用 $\hat{\alpha} = \text{accepted}/(\text{accepted} + \text{rejections})$。一轮中第一次拒绝之后的 draft 从未被检验，不能计入分母。实测 gpt2→gpt2-xl γ=8 时，正确值是 0.72，accepted/proposed 只有 0.28。采样时 $\hat{\alpha}$ 应与 $\mathbb{E}[\sum \min(p, q)]$ 吻合（0.6285 对 0.6423），这可以当作自检。

常见错答：用 accepted/proposed 估计，然后发现公式「不准」。

面试追问：为什么采样版推测解码不改变分布？

因为接受概率和残差分布是配好的：$P(x) = \min(p, q) + (1 - \sum \min(p, q)) \cdot \frac{(p-q)^+}{\sum (p-q)^+} = p(x)$。实测 4000 次采样，对精确 p 的卡方 p 值为 0.657，对 q 则为 8.9×10^{-168}。

常见错答：「拒绝后从 p 重新采样。」这样 x 会被重复计入，分布会偏向 q 与 p 重叠的部分；必须从残差 $(p - q)^+$ 中采样。

面试追问：draft 的 c 为什么不是参数量之比？

因为 batch 1 decode 的时间约等于「权重字节 / 带宽」加上「层数 $\times$ 每层固定开销」，后一项与参数量无关。gpt2 只有 12 层，但每层的开销与大模型相同（约 30 µs），所以 gpt2 单步 0.56 ms，是 gpt2-xl（4.08 ms）的 0.138，而参数量之比是 0.08。要减小 c，可以用更浅的 draft、融合 kernel，或者像 Medusa [8]、EAGLE 那样把 draft 头挂在 target 上。

常见错答：「c 就等于参数量之比。」

面试追问：fp16 下推测解码的贪心输出为什么和 target 不完全一样？

因为数值，而不是算法。验证时 target 一次处理 γ+1 个 token，GEMM 形状与单 token decode 不同，fp16 的累加顺序也就不同。实测 gpt2-xl 的 100 个 prompt 中有 9 个分叉，分叉点 top-2 logit 差最大为 0.0078（fp16 的 ulp）；fp32 下则是 120/120 完全一致。工程上可以用「平局容忍」或 batch-invariant kernel 解决。

常见错答：「推测解码在 fp16 下是有损的。」算法本身无损，差异来自数值，而且同样的差异也会出现在 KV 与无 cache 之间。

面试追问：CUDA Graph 带来了什么，代价是什么？

它把每步上千次 kernel 启动合成一次 replay，gpt2 单步从 1.56 ms 降到 0.49 ms。代价是形状必须固定：位置要做成 device 张量，attention 要在固定长度上加 mask。如果把长度固定为整块 $T_{\max}$，长上下文下 attention 的成本会随 $T_{\max}$ 而不是实际长度增长（P=896 时 graph 用 810.5 ms，反而

慢于 eager 的 794.6 ms）。实际系统会按长度分桶，capture 多个 graph。

常见错答：「CUDA Graph 让 kernel 本身变快。」它只消掉启动和 CPU 端的开销。

面试追问：为什么大 batch 时推测解码收益下降？

因为大 batch 下 decode 逐渐接近计算受限，验证 $\gamma{+}1$ 个 token 不再免费。gpt2-xl 在 $B{=}128$ 时验证 9 个 token 是单 token 的 2.40 倍，代入公式，$\gamma{=}8, T{=}1$ 的预测加速只有 0.674 倍。所以 serving 系统通常在低负载时开推测解码，高负载时关掉或减小 γ。

常见错答：「batch 大了 α 会下降。」α 与 batch 无关，变的是 v 和 c。

面试追问：KV cache 显存怎么算？

$2 \cdot L \cdot B \cdot H_{kv} \cdot d_h \cdot T \cdot$ bytes。gpt2-xl 在 fp16 下每 token 300 KiB，$T{=}1024, B{=}32$ 时为 10.07 GB，与 `memory_allocated` 逐字节相等。Llama-3-8B 用 GQA-8，每 token 只有 128 KiB（`inference_math.csv`）。

常见错答：漏掉因子 2（K 和 V 各一份），或者用 H 代替 H_{kv}。

7 局限

下面这些是本实验没有覆盖、或者数据不足以支撑的地方。

不能推出的结论

- 推测解码只实现了 batch 1。大 batch 下的加速是用实测 $c(B)$、$v(B)$ 与 batch 1 的 $\hat{\alpha}$ 拼出来的预测，不是端到端测量。
- 所有计时都是单卡、fp16、PyTorch 算子级实现，没有 fused kernel、FlashDecoding 或量化。per-kernel 开销的结论针对这种实现，不代表 vLLM [7]、TensorRT-LLM 的绝对性能。
- α 来自 WikiText 的 100 个随机窗口，不同领域（代码、对话）的 α 可能差别很大；GPT-2 系列共享 tokenizer 和训练数据，所以 α 比一般的异构模型对偏高。
- 分布检验只覆盖一个 prompt 的前 3 个 token。它能发现明显的偏差，但不是对所有上下文的证明（证明见第 4.7 节 之后的推导，以及 CPU 版对显式 p, q 做的 60,000 次检验）。
- GPU 与其他 6 个任务共享（用锁串行），计时取 20–30 次的中位数，但仍可能有频率或温度漂移；CPU 部分的计时受共享机器的负载影响。
- fp16 下的贪心一致性不是 bit-exact（gpt2-xl 为 91/100），我们只证明了分叉点是 fp16 平局。

8 结论

读完这份报告，值得带走的是下面四点。

1. KV cache 的收益取决于瓶颈在哪里。CPU 上它省掉的是 FLOP（T=1024 时快 12.82 倍）；GPU batch 1 时，短上下文的瓶颈是 kernel 启动，eager KV 反而更慢，要配合 CUDA Graph 才能真正兑现收益；长上下文时还要注意 decode attention 的并行度（SDPA 加 mask 对 bmm：6.36 对 4.22 ms）。

2. roofline 是下界，不是预测。gpt2-xl batch 1 的实测是下界的 1.9 倍，差值是每层几十微秒的固定开销；大 batch 时比值降到 1.31，剩下的差距在 attention 读 KV 的效率上。
3. 推测解码的公式可以直接用来调参，前提是 α 用正确的估计量，c 和 v 用实测值。有了这三个数，公式对 60 个配置的预测与实测之比为 0.87–1.00，并正确指出了最优的 γ=3–4 和减速区。
4. 正确性要分精度讨论。fp32 下所有「必须一致」的检查都成立；fp16 下的分叉全部发生在 top-2 差 ≤ 1 ulp 的平局处。采样版在真实模型上通过了对精确分布的卡方检验。

参考文献

[1] Y. Leviathan, M. Kalman, Y. Matias. Fast Inference from Transformers via Speculative Decoding. ICML 2023.

[2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, J. Jumper. Accelerating Large Language Model Decoding with Speculative Sampling. arXiv:2302.01318, 2023.

[3] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever. Language Models are Unsupervised Multitask Learners. OpenAI, 2019.

[4] R. Pope et al. Efficiently Scaling Transformer Inference. MLSys 2023.

[5] S. Williams, A. Waterman, D. Patterson. Roofline: An Insightful Visual Performance Model for Multicore Architectures. CACM 2009.

[6] T. Dao, D. Haziza, F. Massa, G. Sizov. Flash-Decoding for Long-Context Inference. 2023.

[7] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023.

[8] T. Cai et al. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. ICML 2024.

[9] S. Merity, C. Xiong, J. Bradbury, R. Socher. Pointer Sentinel Mixture Models. ICLR 2017.